# Reproducing and Improving FiDIP for Infant Pose Estimation on SyRIP

Vishnu Purohitham
EECE 7373 Machine Learning with Small Data, Northeastern University, Fall 2026

Build: 2026-11-06 01:04 EST, commit `54bcdbf` (+uncommitted report changes)

**Abstract**

Infant pose estimation is a small-data problem: privacy limits the number of labelled real images, and adult-trained models transfer poorly to infant bodies and poses. FiDIP [16] fine-tunes a COCO-pretrained DarkPose HRNet-W48 on SyRIP (200 real and 1000 synthetic training images), with a domain classifier that pushes real and synthetic features together. It reports 93.6 AP on SyRIP Test100. We reproduce this result and try to improve on it with an AI coding agent on Northeastern's Explorer cluster. Every evaluation condition is stated, and every model is scored by one audited evaluator.

The released FiDIP checkpoint scores **TBD** AP on Test100. The released training loop selects its checkpoint on Test100. We instead select all models on Val400, the 397 real validate500 images that neither belong to nor duplicate Test100, and evaluate Test100 only for pre-registered configurations. Retraining FiDIP gives **TBD** ± **TBD** AP over **TBD** seeds, against **TBD** ± **TBD** AP without domain adaptation.

Checking the data surfaced issues that change the numbers:

- validate500 ships a constant placeholder object area, which makes OKS meaningless;
- an annotation with id 0 in Test100 is never matched by pycocotools, which caps recall at 0.99 in our numbers and the published ones alike;
- five Test100 images duplicate training images;
- the training file that the README points to carries another dataset's labels for every synthetic image.

A controlled backbone and resolution study in MMPose leads to our pre-registered configuration, **TBD**. It scores **TBD** ± **TBD** AP on Test100, **TBD** AP from the released checkpoint (95% CI [**TBD**, **TBD**]). Every run, decision and failure is logged in the repository.

## 1 Introduction

**The problem.** Long-term observation of infant posture and movement can support early developmental risk assessment. Automating it starts with 2D keypoint estimation from ordinary RGB images [16]. Infants make this a small-data problem in two ways. Privacy limits the number of labelled real images. And infants differ from the adults in COCO [20] in limb proportions, pose repertoire and context, so adult-trained estimators degrade on infant-specific poses [16].

**FiDIP.** FiDIP (Fine-tuned Domain-adapted Infant Pose) [16] addresses both. It fine-tunes a COCO-pretrained pose network on SyRIP [3], a mix of 200 real web images and 1000 synthetic renders, while a domain classifier pushes the features of real and synthetic images together. Its strongest variant, DarkPose [28] with an HRNet-W48 backbone [24], reports 93.6 AP on SyRIP Test100 [2]. As far as we could establish (Section 2.5), this is still the best published number under that protocol.

**Goals.** This report documents a hands-on reproduction of FiDIP and an attempt to improve on it, carried out with an AI coding agent on Northeastern's Explorer cluster. We had four goals:

**(G1)** Reproduce the published baseline, from the released checkpoints and by retraining.
**(G2)** Make the evaluation conditions explicit, and identical for every number we compare.
**(G3)** Improve Test100 AP on the same split, without using Test100 for any decision.
**(G4)** Keep the work reproducible and the agentic process auditable.

**Findings.**

- **Reproduction** (Section 4). The released FiDIP checkpoint scores **TBD** AP on Test100 with our evaluator, against the published 93.6. Retraining with the released recipe gives **TBD** ± **TBD** AP with domain adaptation and **TBD** ± **TBD** AP without it (**TBD** seeds each).
- **Protocol** (Sections 2.3 and 3.3). Run literally, the README training command fine-tunes from random weights with the adaptation weight $\lambda = 0$, and the training loop keeps the epoch that is best on Test100. We select every model on Val400, the 397 validate500 images that neither belong to nor duplicate Test100, and pre-register each configuration before its single Test100 evaluation.
- **Data and evaluation conditions** (Sections 3.3 and 3.4). Hashing, near-duplicate search and label/image consistency checks found six issues that would silently bias results:
  - a constant placeholder object area in validate500;
  - an annotation with id 0 in Test100 that pycocotools never matches, which caps AR at 0.99 and moves AP by **TBD** for the released model;
  - five Test100 images that duplicate training images;
  - three Val400 candidates that duplicate Test100 images;
  - MINI-RGBD labels on the synthetic images in the README's training file;
  - unannotated real images that duplicate validation images.

  MMPose's evaluator lacks the id-0 artifact, so we accept a claim only if it holds under both matching protocols.
- **Improvement** (Section 5). We ran a controlled backbone and input-resolution comparison under one MMPose recipe, plus ablations, all selected on Val400. The pre-registered winner, **TBD**, scores **TBD** ± **TBD** AP on Test100. That is **TBD** AP from the released checkpoint, with a paired-bootstrap 95% CI of [**TBD**, **TBD**].
- **Process** (Sections 8 and 9). The agent kept `SKILL.md`, `AGENT.md` and `MANIFEST.md` current during the work, not after it, and every reported number traces to a run and a commit.

**Roadmap.**

- Section 2: FiDIP and SyRIP, including where the released code differs from the paper.
- Section 3: the cluster workflow, data preparation, split protocol and evaluator.
- Sections 4 to 6: reproduction, improvement and analysis.
- Sections 7 to 9: limitations, lessons learned and AI-tool use.
- The appendix: commands, hyperparameters, compute, provenance and a rubric map.

## 2 Background

### 2.1 Top-down pose estimation and its metric

**Models.** A top-down estimator crops each person box to a fixed input size (e.g., 384×288) and predicts one heatmap per keypoint at 1/4 resolution, from which it decodes coordinates. The models used here:

- SimpleBaseline [26] adds deconvolutions to a ResNet.
- HRNet [24] keeps a high-resolution branch throughout, fused with lower-resolution branches. W48 is that branch's width.
- DARK [28] decodes sub-pixel maxima with a Taylor expansion of the smoothed log-heatmap, and encodes targets without quantisation.
- UDP [15] removes systematic errors from the resizing and flipping transforms and from keypoint encoding.
- ViTPose [27] uses a plain Vision Transformer with a light decoder.

Flip testing averages the heatmaps of the image and of its mirror, with left and right channels swapped.

**Metric.** COCO evaluation [1, 20] scores a prediction against a ground-truth (GT) instance by the object keypoint similarity

$$\mathrm{OKS} = \frac{\sum_i \exp\left(-d_i^2/(2\,s^2\kappa_i^2)\right)\,\delta(v_i > 0)}{\sum_i \delta(v_i > 0)}, \qquad \kappa_i = 2\sigma_i, \tag{1}$$

where $d_i$ is the pixel error of keypoint $i$, $s^2$ is the GT object *area*, $\sigma_i$ are the 17 COCO constants, and $v_i$ is the visibility flag. AP averages, over OKS thresholds $t \in \{0.50, \ldots, 0.95\}$, the area under the precision–recall curve of detections ranked by score. $\mathrm{AP}_{50}$ and $\mathrm{AP}_{75}$ are single thresholds, and AR is the corresponding mean recall. SyRIP is evaluated with GT person boxes and one infant per image, so AP depends only on the per-image OKS and on how the instance scores rank the images.

### 2.2 The SyRIP dataset

**Images.** SyRIP [3, 16] has 700 real images from YouTube and Google Images; the README says the 200 real training images show more than 100 infants. It also has 1000 synthetic images, rendered from the SMIL infant body model [13]. To make each one, SMIL is fitted to a real infant's 2D pose, the pose is perturbed, and the body is rendered with random textures (more than 12), lighting, camera and background [16].

**Splits.** Training uses all 200 real and 1000 synthetic images. validate500 (the paper's *Test500*) holds 500 real images, and its 100 most complex poses form validate100 (*Test100*). Annotations use COCO's 17 keypoints. Section 3.3 describes the overlaps we found between these splits.

**Other infant data.** MINI-RGBD [14], the only earlier public infant dataset, has 12 synthetic sequences of simple poses. A password-protected *expanded* SyRIP adds 400 real training images; Section 7 explains why we do not use it.

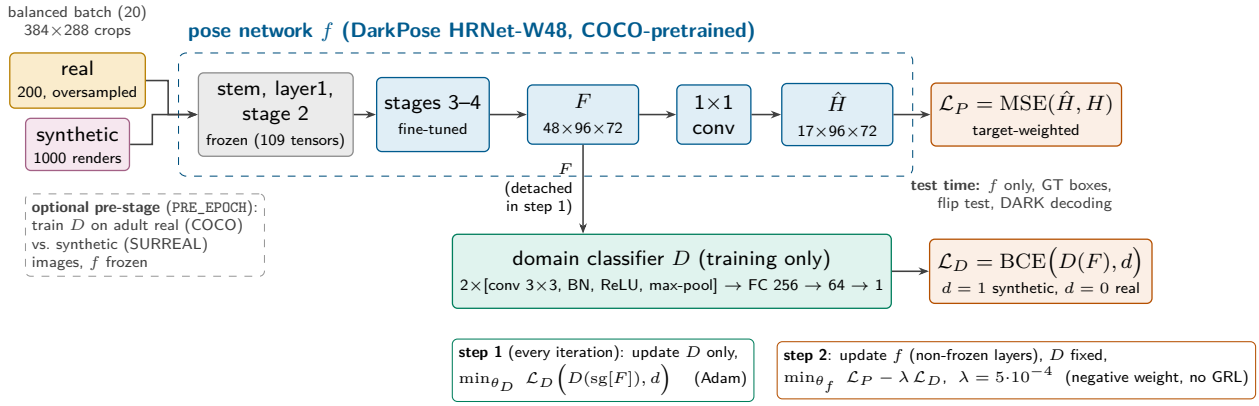



**Figure 1:** FiDIP as released (DarkPose HRNet-W48), from `lib/models/adaptive_pose_hrnet.py` and `lib/core/function.py`. Step 1 of each iteration updates $D$ on detached stage-4 features. Step 2 updates the pose network with $\mathcal{L}_P - \lambda\mathcal{L}_D$, implemented as a negative loss weight. Only the pose network runs at test time.

## 2.3 FiDIP, as published and as released

**As published.** FiDIP [16] attaches a binary real/synthetic domain classifier $D$ to the feature extractor of a COCO-pretrained pose network $f$. Training has two phases:

1. *Initialisation:* $D$ learns to separate adult real images (1904 from COCO val2017) from adult synthetic ones (2000 from SURREAL [25]), with $f$ frozen.
2. *Alternating training:* each iteration has two stages. Stage I updates $D$ with the binary cross-entropy $\mathcal{L}_D$. Stage II updates $f$, with $D$ fixed, to minimise

$$\mathcal{L}(\theta_f, \theta_y, \theta_d) = \mathcal{L}_P(\theta_f, \theta_y) - \lambda\, \mathcal{L}_D(\theta_f, \theta_d), \tag{2}$$

   where $\mathcal{L}_P$ is the heatmap MSE.

The $-\lambda\mathcal{L}_D$ term rewards features that $D$ cannot separate by domain. This is the domain-adversarial idea of Ganin and Lempitsky [10], Ganin et al. [11], which the paper describes as a gradient reversal layer (GRL).

**As released.** We audited the official code [2] at commit `a76b4f4`. Figure 1 shows the released HRNet variant, and Table 1 compares the paper, the released code and our runs. Three differences decide what a reproduction measures:

(i) The infant config sets $\lambda = 0$, so running it literally is plain fine-tuning.
(ii) The config loads no checkpoint: `MODEL.PRETRAINED` is empty, so without a `TRAIN.CHECKPOINT` override training starts from random weights.
(iii) After every epoch the training loop evaluates on `validate_infant` (Test100) and keeps the best epoch as `model_best`, which is selection on the test set.

## 2.4 Published results and where the headroom is

Table 2 copies the published Test100 numbers. With one infant and one detection per image, the true positives at a stricter OKS threshold are a subset of those at a looser one, so $\mathrm{AP}_t$ cannot increase with $t$. DarkPose + FiDIP reports $\mathrm{AP}_{50} = \mathrm{AP}_{75} = 98.5$, so all six thresholds from 0.50 to 0.75 score

**Table 1:** FiDIP as described in the paper (arXiv v5), as released (code and `w48_384x288_adam_lr1e-3_infant.yaml` at `a76b4f4`), and as we ran it.

| Aspect | Paper [16] | Released code / config [2] | Our reproduction |
|---|---|---|---|
| Initialisation of $f$ | COCO-pretrained DarkPose | none unless `TRAIN.CHECKPOINT` is passed (random init) | COCO DarkPose `w48_384x288.pth` |
| $\lambda$ | $5{\cdot}10^{-4}$ | 0 (infant yaml); $5{\cdot}10^{-4}$ only in `_custom.yaml` | $5{\cdot}10^{-4}$; 0 as the fine-tune ablation |
| Optimiser, LR | Adam, $10^{-3}$ | Adam, $10^{-4}$, constant (LR steps at epochs 40 and 200 are never reached); $10^{-3}$ in `_custom.yaml` | released yaml; paper recipe as one extra attempt |
| Epochs / batch | 100 / 64 | 20 / 20 | released yaml |
| Adversarial update | gradient reversal layer | alternating updates with a negative loss weight; no GRL module | as released |
| Domain classifier | three FC layers | 2×[conv3×3, BN, ReLU, max-pool] + FC 256–64–1 on the 48-channel stage-4 high-resolution branch | as released |
| Classifier pre-stage | 1 epoch, batch 128; 1904 COCO vs. 2000 SURREAL adults | `PRE_EPOCH` 0 (infant yaml), 10 (`_custom.yaml`); the adult set (2029 real + 2028 synthetic) is said to be "on request" but ships in `syrip_for_train.zip` | `PRE_EPOCH` 10 on the shipped adult set |
| Frozen layers | Res1–3 (ResNet ablation) | first 109 parameter tensors (stem, `layer1`, stage 2) | as released |
| Real/synthetic balance | domain-dependent loss scale $S(I_i)$ | balanced sampler (200 real oversampled to 1000, interleaved, no reshuffle between epochs); unweighted loss | as released |
| Checkpoint selection | not stated | best Test100 AP, checked every epoch | best Val400 AP epoch |

**Table 2:** Published SyRIP Test100 results [2, 16]: COCO-pretrained ("Base"), fine-tuned on SyRIP, and FiDIP. The published baseline of this report is DarkPose + FiDIP, which also reports $AP_{50} = AP_{75} = 98.5$.

| Method | Backbone | Input | Base AP | + Finetune AP | + FiDIP AP | AR (FiDIP) |
|---|---|---|---|---|---|---|
| SimpleBaseline | ResNet-50 | 384×288 | 82.4 | 90.1 | 91.1 | 92.6 |
| DarkPose | HRNet-W48 | 384×288 | 88.5 | 92.7 | **93.6** | 94.6 |
| Pose-MobileNet | MobileNetV2 [23] | 224×224 | 46.5 | 78.9 | 79.2 | 84.1 |

98.5. The four strict thresholds then average

$$AP_{80:95} = (10 \times 93.6 - 6 \times 98.5)/4 \approx 86.3.$$

The same calculation gives 84.0 for DarkPose + Finetune (92.7 AP, also 98.5/98.5). FiDIP's 0.9-point gain therefore lies entirely at strict thresholds, and so must any further gain. Section 3.4 shows that 98.5 is essentially a perfect score at loose thresholds on this test set.

## 2.5 Related work on SyRIP

We found no later paper that beats 93.6 on Test100 with GT boxes. Later work on SyRIP uses other protocols:

- Gama et al. [9] benchmark seven adult models zero-shot on the 500 real test images with *detected* boxes; ViTPose is best, at 41.9 AP.
- SHIFT [6] adapts from SURREAL adults without infant labels and reports PCK@0.05.

- Huang et al. [17] and Zhou et al. [29] evaluate 3D pose on Test100.
- Grafton et al. [12] and Jahn et al. [19] use SyRIP only for cross-dataset checks.
- AggPose [7] and the InfantNet preprint [18] evaluate on their own, larger infant datasets.

# 3 Implementation

## 3.1 Compute and agentic workflow

**Compute.** Training and evaluation ran under Slurm on Northeastern's Explorer cluster [22]:

- `gpu` (H200/A100, 8 h limit) for training;
- `sharing` (1 h) for smoke tests and evaluations;
- `short` for CPU jobs.

Datasets, environments and checkpoints stay on `/scratch`; only code, metrics, predictions and figures travel to the laptop, which had about 7 GB free.

**Agents.** Claude Code [5], running Claude Opus 5.5, acted as an *orchestrator* (Figure 2). After the student approved a written plan, it launched four parallel workers, each with a self-contained brief:

- *repro*: data, environments, released checkpoints and retraining;
- *improve*: MMPose experiments;
- *analysis*: the evaluator, statistics, figures and generated tables;
- *report*: this document and the README.

A fresh-context reviewer checks the final state against the rubric.

**Live files.** The orchestrator owns three of them:

- `SKILL.md`: how to operate the cluster;
- `AGENT.md`: the lab notebook of asks, attempts, failures and dated decisions D1, D2, ...;
- `MANIFEST.md`: datasets, checkpoints, configs, and the run behind every number.

Workers log commands, job IDs and errors in `notebook/<stream>.md`, and register runs in `results/registry.csv`.

**Shared rules.** All agents load `CLAUDE.md`, which requires:

- no compute on login nodes;
- no selection on Test100;
- one evaluator for every number;
- secrets only in a git-ignored `.env`;
- timestamps from `date`.

**Cluster loop.** The agent connects with `ssh -o BatchMode=yes -o ConnectTimeout=15` and wraps slow login-node commands in `timeout`, so failures surface quickly. For each job it:

1. pushes code with tar over ssh;
2. submits an `sbatch` script, smoke-tested first on `sharing`;
3. polls `squeue`, `sacct` and the logs;
4. pulls back only small artifacts.

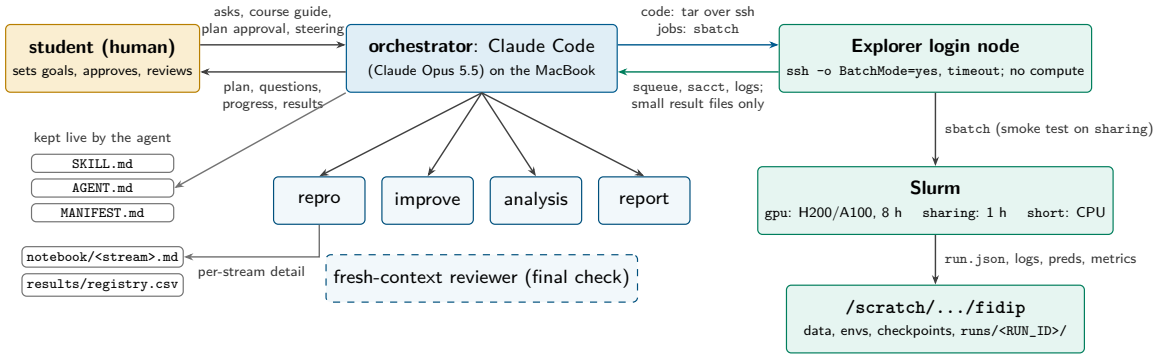



**Figure 2:** Agentic workflow. The student sets goals, supplies the course material and approves the plan. The orchestrator keeps the three live files current and delegates to parallel workers. All compute goes through Slurm, and only small artifacts return to the laptop.

Each run's directory `runs/<RUN_ID>/` holds its resolved config, logs, predictions, evaluator outputs, and a `run.json` with the commit and tree SHAs, Slurm job ID, node, GPU, seed and checkpoint SHA256.

### 3.2 Environments and compatibility patches

**Environments.** The upstream `fidip_env.yml` cannot be built: its pip stage pins `grpcio==1.48.2`, which has no Python 3.12 wheel, and `mkl-*` packages with no wheels. We built three environments instead:

- *FiDIP:* pip-only, Python 3.12, PyTorch 2.3.1 (CUDA 12.1), NumPy 1.26.4, with every imported package pinned to the yml's version.
- *MMPose:* Python 3.10, PyTorch 2.1.2, MMEngine 0.10.7, MMCV 2.1.0, MMPretrain 1.2.0 and MMPose 1.3.2 [21], from conda-forge, with `numpy<2` and `opencv-python<4.12`.
- *Analysis:* CPU-only, with the same NumPy and pycocotools pins as FiDIP.

Package lists are in `envs/` and checkpoint hashes in `MANIFEST.md`.

**Upstream patches.** Appendix B lists six upstream traps and our patches. The most consequential trap, the README training command, needs no code change: `TRAIN.CHECKPOINT` and `TRAIN.LAMBDA` are passed on the command line. The course guide's `gpu_test.py`, `tools/demo.py` and `tools/streamlit_demo.py` (and `fidip_simple.yml`) do not exist upstream. We wrote the GPU check and both demos from the guide's documented interface.

### 3.3 Data preparation and the split protocol

**Download.** A Slurm job (`scripts/get_data.sh`):

1. downloads both public SyRIP archives, which needed no password;
2. verifies their SHA256;
3. extracts them on a compute node, because the login node killed `unzip`;
4. checks file counts against the archives;
5. builds the upstream layout from symlinks and writes the split files.

The adult `train_pre_infant` set (2029 real and 2028 synthetic images), described upstream as available on request, ships inside `syrip_for_train.zip`, so the classifier pre-stage can be run

faithfully. `SyRIP.zip`'s `images/train_infant/` also holds 300 unannotated real images, 199 of them byte-identical to validate500 images. We use only images listed in an annotation file.

**Training labels.** Two annotation files cover the 1200 training images: the official `200R_1000S` file in `SyRIP.zip`, and `train_infant.json` in `syrip_for_train.zip`. They agree on the real images but differ on every synthetic one, because the second file carries MINI-RGBD labels:

- its source fields point to MINI-RGBD frames;
- eyes and ears are zeroed on most entries;
- some sizes and boxes do not fit the renders, with a median keypoint offset of 191 px from the official labels (`notebook/repro.md`).

The images in the two archives are identical, except that the second archive's `validate_infant/test11.jpg` is a copy of `test24.jpg`. Following the README literally therefore trains on mislabelled synthetic data and scores one test image against the wrong picture: the released checkpoint scores **TBD** AP on that archive's test images, against **TBD** AP on the correct ones. We use `SyRIP.zip`'s images and `200R_1000S` labels everywhere (D17); attempt R020 measures what the README-literal path costs.

**Val400 (D15).** validate100 is a subset of validate500 (image ids 0–99, with identical files). **Val400** is validate500 ids 100–499 minus three images that duplicate Test100 images (two exact copies, one near-duplicate), which leaves 397 images. Exact copies were found by file hash, and near-duplicates by a flip-aware perceptual hash with visual confirmation (`scripts/near_dups.py`).

**Train/test overlap (D16).** Test100 images `test20`, `test42`, `test46` and `test50` are byte-identical to training images 73, 315, 272 and 340, and `test7` is a near-duplicate of training image 47. We keep Test100 as shipped, for comparability with the published numbers (which carry the same overlap). As a sensitivity check we also report the 95-image *Test100-clean* subset, where the released model's AP changes by **TBD**. Two further properties of the shipped GT need care.

**Placeholder areas (D8).** Every validate500 annotation carries the same `area`, 2443298.48 ($\sqrt{\text{area}} \approx 1563$ px), and the training file carries another constant, 14232.77; validate100 instead uses area $= w \cdot h$ of its box. Since OKS divides squared errors by the area (Equation (1)), the shipped Val400 makes almost any prediction look perfect: the released model scores **TBD** AP on it. We rebuilt Val400, validate500 and the framework-neutral training file with area $= w \cdot h$ from their own boxes, and left Test100 exactly as shipped. `scripts/make_splits.py` asserts this rule, and the evaluator refuses GT files whose areas are all identical.

**Different boxes.** On the 100 shared images, validate100's boxes are a median **TBD**% larger than validate500's, while the keypoints agree on 98 images; the other two were re-annotated. Val400 boxes are therefore tighter than Test100's, which means a smaller OKS scale and tighter crops (Section 6.1).

**Protocol (D1).** Figure 3 summarises the protocol:

- Every choice of epoch, hyperparameter, backbone or recipe is made on Val400.
- A configuration reaches Test100 only after a commit lists it, with its Val400 evidence and checkpoint rule, in `results/PREREGISTRATION.md`.

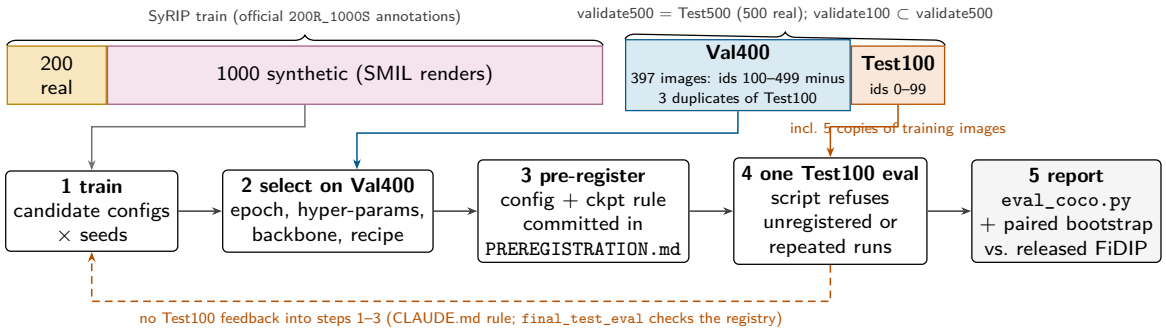



**Figure 3:** Split and selection protocol (bar widths proportional to image counts). Test100 is read once per pre-registered configuration, and nothing learned from it flows back into training or selection.

- The final-evaluation script refuses unregistered or repeated runs, and checks the checkpoint SHA256 against `run.json`.
- The released checkpoints and the official-recipe retrains are reproduction targets, so they are evaluated on Test100 directly; the retrains' epochs are still chosen on Val400.

**Why Val400.** It keeps the training data identical to the paper's, and it is almost twice as large as the real training set; cross-validation on the 200 real images could not match that without shrinking training. It has two costs: Test500 is no longer clean, so we never report it, and Val400 is not the complex-pose subset, so it may rank models differently from Test100 (Section 6.1).

### 3.4 One evaluator for every number

**Pipeline.** Every metric comes from `eval/eval_coco.py` (v1.0) applied to a prediction JSON, and both frameworks take the same path:

1. a loader that accepts twelve prediction encodings;
2. one rescoring rule, score = mean($c_k \mid c_k > 0.2$), which is upstream FiDIP's rule and MMPose's `bbox_keypoint` mode with a box score of 1 for GT boxes. It keeps the image ranking, and therefore AP, comparable;
3. a verbatim port of upstream OKS-NMS, which is a no-op with one box per image;
4. pycocotools 2.0.8 `COCOeval` [1], with the COCO sigmas and the GT area.

Besides the usual metrics it writes per-threshold AP, $AP_{80:95}$ and per-image OKS. It also writes per-keypoint errors normalised by $\sqrt{wh}$ of the GT box (the OKS scale on Test100), with PCK@{0.05, 0.1, 0.2}.

**The annotation-id-0 artifact (D10).** In Test100, annotation ids equal image ids, so `test0.jpg` carries id 0. `COCOeval.evaluateImg` stores the matched GT id in `dtMatches`, and `accumulate` treats a stored 0 as "unmatched". The infant in `test0.jpg` can therefore never be a true positive, and its detection always counts as a false positive. This caps AR at 0.99, and even a perfect model's AP at every threshold is

$$\frac{r + 0.99\,(100 - r)}{101} \in [0.980,\ 0.990],$$

where $r$ is the rank of test0's score (unit-tested). The published $AP_{50} = AP_{75} = 98.5$ falls inside this band, so on the 99 matchable images the published models are already perfect at loose thresholds.

The near-constant $AR_{50} = 99.0$ of the upstream table has the same cause.

The size of this penalty depends only on where test0's score ranks: between **TBD** and **TBD** AP at realistic accuracy. Our headline `paper` protocol keeps the ids as shipped, so it stays comparable with 93.6. A `fixed` protocol shifts the ids off 0. An improvement claim counts only if its paired CI excludes 0 under *both* protocols, and every model's test0 rank is reported (D13). On Val400, which has no id 0, the two protocols coincide. This is a property of the evaluation tooling, not an error in the published work: anyone scoring this file with pycocotools gets the same cap.

**MMPose's evaluator.** MMPose's `CocoMetric` uses xtcocotools [4], which marks unmatched detections with $-1$. Its printed Test100 AP therefore equals our `fixed` protocol, **TBD** AP away from `paper` on identical predictions. Comparing it with 93.6 would overstate any gain, which is why only `eval_coco.py` numbers are compared (D2).

**Verification and statistics.**

- *Unit tests.* 24 tests cover the GT-as-prediction case, the id-0 ceiling, noise monotonicity, encoding invariance, rescoring, OKS against `COCOeval` internals, and NMS against upstream (Appendix C).
- *Cross-check.* On the released checkpoint's predictions, upstream's own test script reports **TBD** AP and ours **TBD** AP.
- *Uncertainty.* `eval/bootstrap_ci.py` resamples the 100 test images ($B =$ **TBD**), recomputes AP exactly, and reports paired 95% percentile CIs of seed-averaged AP differences and Holm-adjusted one-sided $p$-values [8].

# 4 Reproduction results

## 4.1 Released checkpoints

**Setup.** We evaluated three checkpoints under their released test settings (GT boxes, flip test, DARK post-processing), with predictions scored by `eval_coco.py` (Table 3, upper block):

- `hrnet_fidip.pth`, DarkPose + FiDIP;
- `mobile_fidip.pth`, Pose-MobileNet + FiDIP;
- the COCO-only DarkPose `w48_384x288.pth` that FiDIP starts from.

**Results.** The released FiDIP model scores **TBD** AP under the `paper` protocol (**TBD** from the published 93.6), and **TBD** AP under `fixed`. The individual metrics, against the published values:

- $AP_{50}$ **TBD** and $AP_{75}$ **TBD** (98.5 each);
- AR **TBD** (94.6);
- $AP_{80:95}$ **TBD** (86.3 implied, Section 2.4).

The released Pose-MobileNet scores **TBD** AP (published 79.2), and the COCO-only DarkPose **TBD** AP (published 88.5).

***Interpretation pending final results:*** *Does the released checkpoint reproduce 93.6 within what 100 test images allow? If not, does $AP_{50}/AP_{75}$ or $AP_{80:95}$ explain the gap, and does upstream's printed AP agree (Section 3.4)?*

**Table 3:** Reproduction on SyRIP Test100 (`paper` protocol, GT boxes, flip test) and Val400. *Upper block:* released checkpoints scored by `eval_coco.py`. *Lower block:* retrains with the released recipe from COCO DarkPose weights (mean ± s.d. over seeds, epochs selected on Val400). "Publ." is the published AP [2, 16].

| | | SyRIP Test100 | | | | | | | Val400 |
|---|---|---|---|---|---|---|---|---|---|
| Model | Weights / runs | Publ. AP | AP | $AP_{50}$ | $AP_{75}$ | $AP_{80:95}$ | AR | $AP^{fixed}$ | AP |
| DarkPose W48 (COCO only) | `w48_384x288.pth` | 88.5 | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |
| DarkPose + FiDIP | `hrnet_fidip.pth` | 93.6 | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |
| Pose-MobileNet + FiDIP | `mobile_fidip.pth` | 79.2 | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |
| DarkPose + Finetune ($\lambda$=0) | retrain, **TBD** seeds | 92.7 | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |
| DarkPose + FiDIP ($\lambda$=$5{\cdot}10^{-4}$) | retrain, **TBD** seeds | 93.6 | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |

**Pending:** figure `curves_fidip`.{pdf,png} — per-epoch training loss, Val400 AP (and domain loss) for the FiDIP and fine-tune retrains, all seeds

**Figure 4:** Retrain curves: training loss and Val400 AP per epoch for all seeds, with the Val400-selected epoch marked. Test100 was never evaluated per epoch.

## 4.2 Retraining FiDIP and the fine-tuning ablation

**Recipe.** We retrained with the released code and yaml, adding two command-line overrides (D5): `TRAIN.CHECKPOINT` set to the COCO DarkPose weights, and `TRAIN.LAMBDA` set to the paper's $5{\cdot}10^{-4}$. The classifier pre-stage runs for 10 epochs (`PRE_EPOCH`, as in `_custom.yaml`) on the shipped adult set. The model is evaluated on Val400 after every epoch, and the best epoch is kept. To isolate domain adaptation, we also ran the same recipe with $\lambda = 0$, which is the paper's "DarkPose + Finetune" row. Each arm has **TBD** seeds.

Because the paper's recipe differs from the yaml (Table 1), we also ran two extra attempts, listed in Section 5.3:

- the paper's recipe once (Adam $10^{-3}$, 100 epochs, batch 64), D12;
- a single-seed $\lambda$ sweep on Val400, D9.

**Results.** On Test100, FiDIP scores **TBD** ± **TBD** AP and fine-tuning **TBD** ± **TBD** AP. On Val400 they score **TBD** ± **TBD** and **TBD** ± **TBD**. The seed-averaged difference on Test100 is **TBD** AP, with 95% CI [**TBD**, **TBD**] and one-sided $p =$ **TBD** (Table 3, lower block).

***Interpretation pending final results:*** *Is FiDIP's published 0.9-AP gain over fine-tuning (92.7 to 93.6) reproduced, and is it distinguishable from seed and sampling noise at $n = 100$? How does the retrain compare with the released checkpoint?*

## 4.3 Training dynamics: over- and underfitting

**What we monitor.** As the course guide asks, every run logs the training loss and the Val400 AP after each epoch, and FiDIP runs also log the domain-classifier loss (Figure 4). Out of **TBD** epochs, the median Val400-selected epoch is **TBD** for FiDIP and **TBD** for fine-tuning.

**Risk factors.** Two features of the data invite overfitting: there are only 200 real images, each seen about five times per epoch through the balanced sampler, and the batch order never changes between epochs. The released recipe counteracts this with a low constant LR ($10^{-4}$) and frozen early layers.

***Interpretation pending final results:*** *Does Val400 AP plateau or fall while the training loss keeps decreasing (overfitting)? Is it still rising at the last epoch (underfitting, which would argue for the paper's longer schedule)? How does the domain loss behave at* $\lambda = 5{\cdot}10^{-4}$*?*

# 5 Improving on FiDIP

## 5.1 Hypothesis and method

**Hypothesis.** The published model is already essentially perfect at OKS 0.50–0.75 on the 99 matchable images (Sections 2.4 and 3.4). Any gain must therefore come from more precise localisation, which $AP_{80:95}$ measures. Two standard levers act on precision: backbone capacity with stronger pre-training, and higher input resolution, which gives finer heatmaps. We test both with ViTPose [27]. In the MMPose model zoo [21], its COCO checkpoints score 78.2 AP (L) and 78.8 AP (H), against 77.2 AP for HRNet-W48 + DARK at 384×288 (COCO val2017, detected boxes).

**Arms.** All improvement runs use MMPose 1.3.2.

- The *control* is HRNet-W48 + DARK at 384×288, the MMPose counterpart of FiDIP's network.
- The *resolution arm* is ViTPose-L at 256×192 and 384×288.
- The *capacity arm* is ViTPose-H at 384×288.

**What is held fixed.** Every arm uses the same training data (200 real + 1000 synthetic, official labels) and FiDIP's 1:1 real:synthetic mix, in which each real image appears five times per 2000-sample epoch. Every arm also shares the epoch budget, warm-up and LR decay, MMPose's COCO augmentation, GT boxes, flip test, and per-epoch Val400 checkpoint selection.

**What varies.** Only what each family needs: HRNet uses Adam, while ViT uses AdamW with layer-wise LR decay and drop-path. Batch size and codec (DARK versus UDP) also differ. We therefore compare backbone families, each with its standard optimiser, rather than doing a pure weight swap.

Starting from FiDIP's nominal LR of $10^{-4}$, we explored recipe knobs (LR, rotation range, schedule, EMA) on Val400 only (Table 6); the resolved settings are in Table 12. Besides SyRIP, the only pose data are the COCO checkpoints. We excluded models whose pre-training adds other pose datasets.

## 5.2 Controlled comparison and ablations

Table 4 gives Val400 AP (mean ± s.d. over seeds) and $AP_{80:95}$ per arm. The MMPose control reaches **TBD** ± **TBD** AP, and the configuration finally pre-registered, **TBD**, reaches **TBD** ± **TBD** AP.

Table 5 changes one factor at a time against a stated default, on Val400 only:

- real-only versus real + synthetic training data;
- the rotation range;
- flip testing;
- the FiDIP weight $\lambda$.

***Interpretation pending final results:*** *Name the Val400 winner. Which matters more, resolution (L at* 256×192 *vs.* 384×288*) or capacity (L vs. H)? Does the gain appear in* $AP_{80:95}$ *as hypothesised? Is synthetic data still useful for a strong backbone, and what do rotation, flip-TTA and* $\lambda$ *contribute?*

**Table 4:** Controlled comparison under one MMPose recipe (Val400, mean ± s.d. over seeds). Test100 is shown only for pre-registered runs (otherwise —). GPU-hours are per training run.

| Backbone | Input | Params (M) | Val400 AP | Val400 $AP_{80:95}$ | Seeds | Test100 AP | GPU-h/run |
|---|---|---|---|---|---|---|---|
| HRNet-W48 + DARK (control) | 384×288 | TBD | TBD | TBD | TBD | TBD | TBD |
| ViTPose-L | 256×192 | TBD | TBD | TBD | TBD | TBD | TBD |
| ViTPose-L | 384×288 | TBD | TBD | TBD | TBD | TBD | TBD |
| ViTPose-H | 384×288 | TBD | TBD | TBD | TBD | TBD | TBD |

**Table 5:** Ablations, one factor at a time, on Val400. Δ is relative to the default row.

| Factor | Setting | Applied to | Seeds | Val400 AP | Δ |
|---|---|---|---|---|---|
| Training data | real + synthetic (default) | TBD | TBD | TBD | — |
| | real only (200) | TBD | TBD | TBD | TBD |
| Rotation aug. | default | TBD | TBD | TBD | — |
| | TBD | TBD | TBD | TBD | TBD |
| Flip test (TTA) | on (default) | TBD | TBD | TBD | — |
| | off | TBD | TBD | TBD | TBD |
| FiDIP $\lambda$ | $5{\cdot}10^{-4}$ (default) | TBD | TBD | TBD | — |
| | TBD | TBD | TBD | TBD | TBD |

### 5.3 All attempts

The course asks for the accuracy of *every* attempt. Table 6 lists all TBD in chronological order, including those that did not help:

- the reproduction variants;
- the domain-adaptation tweaks (the $\lambda$ sweep and the paper recipe);
- every backbone, resolution, augmentation and recipe change.

TBD attempts beat the control's mean Val400 AP. Test100 values appear only for pre-registered runs.

### 5.4 Final evaluation: repeats and confidence intervals

After the Val400 study, TBD was pre-registered at commit TBD, with the checkpoint rule "TBD". It was then evaluated once on Test100 for each of its TBD seeds (Table 7). Differences are paired with the released FiDIP checkpoint on the same 100 images. The published 93.6 has no per-image predictions, so it cannot be paired. The paired image bootstrap of the seed-averaged AP (Section 3.4) covers both test-sampling and seed variation. Following D13, the claim requires the CI to exclude 0 under both the `paper` and the `fixed` protocol (Table 8).

**Published baseline vs. our result, and the exact evaluation conditions**

**Published baseline.** DarkPose + FiDIP, HRNet-W48, 384×288: **93.6 AP** on SyRIP Test100 ($AP_{50}$ 98.5, $AP_{75}$ 98.5, AR 94.6) [2, 16].
**The same model under our evaluator.** Released `hrnet_fidip.pth`: TBD AP (`paper`), TBD AP (`fixed`).
**Our result.** TBD: TBD ± TBD **AP** on Test100, mean ± s.d. over TBD seeds, `paper` protocol.

- Against the released checkpoint on the same images: ΔAP = TBD, 95% CI [TBD, TBD], one-sided $p$ = TBD.
- Against the published 93.6: TBD.

**Table 6:** Every attempt with its Val400 accuracy and outcome, generated from `results/registry.csv`. Run IDs resolve to `runs/<RUN_ID>/` and to `MANIFEST.md`.

| Run ID(s) | Stream | Change vs. control | Seeds | Val400 AP | Δ Val400 | Test100 AP | Outcome |
|---|---|---|---|---|---|---|---|
| TBD | repro | TBD | TBD | TBD | TBD | TBD | TBD |
| TBD | repro (DA) | TBD | TBD | TBD | TBD | — | TBD |
| TBD | improve | TBD | TBD | TBD | TBD | — | TBD |

**Table 7:** Final Test100 comparison (`paper` protocol). ΔAP is paired with the released checkpoint on the same images (95% percentile bootstrap CI, $B =$ TBD); $p$ is one-sided and Holm-adjusted. The published row is copied from [2].

| Model | Seeds | AP | $AP_{50}$ | $AP_{75}$ | $AP_{80:95}$ | AR | ΔAP vs. released [95% CI] | $p$ |
|---|---|---|---|---|---|---|---|---|
| DarkPose + FiDIP, published | — | 93.6 | 98.5 | 98.5 | — | 94.6 | — | — |
| DarkPose + FiDIP, released ckpt | — | TBD | TBD | TBD | TBD | TBD | 0 (reference) | — |
| DarkPose + FiDIP, our retrain | TBD | TBD | TBD | TBD | TBD | TBD | TBD | TBD |
| Best configuration (TBD) | TBD | TBD | TBD | TBD | TBD | TBD | TBD | TBD |

- Same contrast under the `fixed` protocol: TBD, 95% CI [TBD, TBD]. Robust under both protocols (D13): TBD.
- $AP_{80:95}$: TBD (TBD against the released checkpoint). On Test100-clean: TBD AP.

**Evaluation conditions**, identical for every compared row:

- *Test set:* SyRIP `validate100` (= Test100), 100 real images exactly as distributed. Boxes, areas and annotation ids are unchanged, and the five images that duplicate training images are kept, as in the published evaluation; Test100-clean (95 images) is a sensitivity check.
- *Training data:* 200 real + 1000 synthetic images, with the official `200R_1000S` labels. No Val400 or Test100 image was added to training.
- *Inference:* GT person boxes, flip test TBD, input TBD, one model per seed. Ensembles are reported separately.
- *Metric:* `eval/eval_coco.py` v1.0, i.e. pycocotools 2.0.8 `COCOeval` (keypoints, COCO-17 sigmas, GT area, maxDets 20) after shared rescoring and OKS-NMS, with annotation id 0 kept as in the published numbers (Section 3.4). Claims must also hold under the `fixed` protocol. The test0 score rank is TBD for our model and TBD for the released one.
- *Selection:* epochs, hyperparameters and the configuration were chosen on Val400 only. The configuration was pre-registered at commit TBD before its single Test100 evaluation, and no Test100 result fed back into any choice.

***Interpretation pending final results:*** *Two-sentence conclusion: is the improvement over the released checkpoint supported by the CI, and where does it come from ($AP_{80:95}$, per-keypoint results in Section 6)? If the CI includes 0, say plainly that no improvement was established.*

**Ensemble (secondary claim).** A coordinate ensemble across seeds (`eval/ensemble_preds.py`) was registered separately and is never mixed with single-model numbers. Its status is TBD. When run, it scores TBD AP, with a paired 95% CI against the released checkpoint of [TBD, TBD].

**Table 8:** Paired image-level bootstrap contrasts on Test100 under both protocols. "Robust" means the 95% CI excludes 0 under both `paper` and `fixed` (D13); W/T/L counts per-image OKS wins, ties and losses.

| Contrast | ΔAP | 95% CI | $p$ | $p_{\text{Holm}}$ | fixed: Δ [CI] | robust | W/T/L |
|---|---|---|---|---|---|---|---|
| best vs. released FiDIP | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |
| FiDIP retrain vs. fine-tune | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |

**Pending:** figure `oks_thresholds`.{pdf,png} — AP at OKS 0.50–0.95: released FiDIP, retrain, best config; paper-protocol ceiling band

**Pending:** figure `val_vs_test`.{pdf,png} — Val400 AP vs Test100 AP, one point per run that has both

**Figure 5:** Left: Test100 AP at each OKS threshold (`paper` protocol); the band is the ceiling of a perfect model under the id-0 artifact. Right: Val400 against Test100 AP for every run evaluated on both sets.

# 6 Analysis

## 6.1 Where AP is lost, and how Val400 relates to Test100

**Per-threshold AP.** Figure 5 (left) plots Test100 AP at each OKS threshold for the released checkpoint, our FiDIP retrain and the best configuration, together with the `paper`-protocol ceiling of a perfect model (Section 3.4). Curves that stay on the ceiling up to $t = 0.75$ and separate only above it confirm that localisation precision, not gross failure, decides AP on Test100.

**Val400 versus Test100.** Figure 5 (right) compares Val400 and Test100 AP for every run evaluated on both. Across these **TBD** runs, the Spearman rank correlation is **TBD**, and Test100 differs from Val400 by **TBD** AP on average. The two sets differ in three ways: Test100 was chosen for complex poses, it is a quarter of the size, and its boxes are looser (Section 3.3). If the ranking transfers anyway, that is evidence that the selection set worked without touching the test set.

**Evaluation choices.** Table 9 keeps the predictions fixed and changes one evaluation choice at a time:

- the matching protocol, which moves the released model by **TBD** AP;
- Test100 versus Test100-clean, which differ by **TBD** AP;
- the GT boxes for the same images, validate100 versus validate500 (**TBD** versus **TBD** AP);
- the shipped placeholder area versus area $= wh$ on Val400;
- pycocotools versus xtcocotools.

Each effect is comparable to FiDIP's published gain over fine-tuning (0.9 AP), which is why all of them are held fixed.

## 6.2 Per-keypoint accuracy

Figure 6 compares the released FiDIP model with the best configuration joint by joint, using median error as a fraction of $\sqrt{wh}$ and PCK@0.05. Table 13 in the appendix gives the numbers.

***Interpretation pending final results:*** *Which joints dominate the error: the distal joints (wrists, ankles), or the hips, which are ambiguous under clothing and in prone poses? Where does the best configuration gain? Compare with the paper's observation that the lower body is hardest for adult models.*

**Table 9:** Sensitivity of AP to evaluation choices, with predictions held fixed. Headline numbers use the first row of each pair.

| Condition | Variant | Split | Released FiDIP | Best config. |
|---|---|---|---|---|
| Matching protocol (D10, D13) | `paper`: ids as shipped (headline) | Test100 | TBD | TBD |
| | `fixed`: ids shifted off 0 | Test100 | TBD | TBD |
| Train/test overlap (D16) | Test100 as shipped (100) | Test100 | TBD | TBD |
| | Test100-clean (95; copies of training images removed) | Test100-clean | TBD | TBD |
| Test images (D17) | `SyRIP.zip` (ours) | Test100 | TBD | TBD |
| | `syrip_for_train.zip` (test11 = copy of test24) | Test100 | TBD | — |
| GT boxes of the same 100 images | validate100 (as shipped) | Test100 | TBD | TBD |
| | validate500 (area = $wh$) | Test100 | TBD | TBD |
| OKS area on Val400 (D8) | shipped constant 2443298.48 | Val400 | TBD | TBD |
| | area = $wh$ (ours) | Val400 | TBD | TBD |
| Metric code | `eval_coco.py` (pycocotools 2.0.8) | Test100 | TBD | TBD |
| | MMPose `CocoMetric` (xtcocotools) | Test100 | TBD | TBD |

**Pending:** figure `per_keypoint`.{pdf,png} — per-keypoint normalised error and PCK@0.05, released FiDIP vs best config (Test100)

**Figure 6:** Per-keypoint error on Test100, normalised by $\sqrt{wh}$ of the GT box (the OKS scale), for the released FiDIP model and the best configuration.

### 6.3 Successes, failures and a failure taxonomy

**Qualitative cases.** Figure 7 overlays the GT and both models' predictions on Test100 images ranked by per-image OKS: the best cases, the worst cases, and those where the two models disagree most.

**Automatic taxonomy.** `eval/taxonomy.py` assigns failure modes with unit-tested rules rather than by eye:

- *left/right swap*: swapping a pair of predicted joints to the opposite GT joints more than halves their summed error and raises OKS by more than 0.02, for well-separated pairs only;
- *whole-body mirror*: mirroring every left/right label raises OKS, which indicates front/back confusion;
- *occlusion*: the number of joints labelled but not visible ($v = 1$);
- *orientation*: the in-plane torso angle (upright, sideways, inverted), and a front/back/profile cue from the shoulder–hip cross product;
- *scale*: the infant's size in the image.

Table 10 counts each mode for both models, and its "Rule" column gives the exact thresholds.

***Interpretation pending final results:*** *Which failure mode is most common and which is most costly (left/right confusion in prone or side-lying poses, inverted torsos, self-occluded limbs)? Does the better model fix one mode, or improve uniformly?*

### 6.4 Inference demo

Part 6 of the course guide asks for two demos:

- a command-line demo, `tools/demo.py`, with the guide's interface: `--cfg`, `--image`, `--model` and `--output`, and optionally `--conf-threshold` and `--output-size`;

**Pending:** figure `qual_success`.{pdf,png} — overlay grid: best Test100 cases (GT vs released FiDIP vs best)

**Pending:** figure `qual_failure`.{pdf,png} — overlay grid: worst Test100 cases and largest disagreements

**Figure 7:** Qualitative Test100 results. Top: the highest per-image OKS. Bottom: the lowest OKS, and the largest disagreements between the two models. Colours are given in the legend.

**Table 10:** Automatic failure taxonomy on Test100 (`eval/taxonomy.py`). Counts are images, and mean OKS is taken within each group.

| | | # images | | mean OKS | |
|---|---|---|---|---|---|
| Failure mode | Rule | Released | Best | Released | Best |
| Left/right swap | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |
| Whole-body mirror | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |
| Heavy occlusion | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |
| Inverted torso | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |
| Low OKS overall | **TBD** | **TBD** | **TBD** | **TBD** | **TBD** |

- a Streamlit app, `tools/streamlit_demo.py`, which uploads an image, sets the confidence threshold with a slider, and shows the predicted keypoints with their confidences.

Figure 8 shows both; the images are in `results/figures/demo/`. The demos run on images without ground truth, so they illustrate behaviour and are not used for any metric.

# 7 Limitations

- **A 100-image test set.** One image can move AP by about a point, and strong models' $AP_{50}/AP_{75}$ sit at the id-0 ceiling. Differences below about one AP are hard to establish (see Table 7), and three seeds give only a rough estimate of seed variance.
- **One protocol.** Like the published work, we use GT person boxes. The numbers therefore say nothing about deployment with a person detector, where zero-shot adult models reach about 42 AP on SyRIP's real test images [9].
- **Selection-set mismatch.** Val400 is what remains of validate500 once the complex poses are removed, and its boxes are tighter than Test100's. It may favour models that do well on easier poses. The correlation in Section 6.1 measures how well it transfers, but evaluating Test100 only once per configuration limits the number of such comparisons.
- **What "reproduction" means here.** Our retrains differ from the released code in three ways:
  - They select epochs on Val400, whereas the released loop selects on Test100. Test-set selection can only raise the reported test score, so a small shortfall of a Val400-selected retrain is expected and is not by itself a failed reproduction.
  - The upstream README points training to `syrip_for_train.zip`, whose synthetic labels do not match the images (Section 3.3). We used the official labels, and we cannot tell which file produced the published models.
  - The paper's recipe differs from the config, and we ran the paper's recipe only once.
- **Kept artifacts.** For comparability, the headline numbers keep two properties of the published evaluation:
  - the pycocotools id-0 artifact. The `fixed` numbers are closer to the true COCO AP, and claims

**Pending:** figure `demo_cli`.{pdf,png} — output image of the command-line demo

**Pending:** figure `demo_streamlit`.{pdf,png} — screenshot of the Streamlit demo app

**Figure 8:** Inference demos. Left: the command-line demo's output with keypoint confidences. Right: the Streamlit app.

must hold under both protocols;
  - the five Test100 images that duplicate training images. They favour any model that memorises its training set, so the Test100-clean subset is reported alongside.
- **Scope of the search.**
  - We tried ViTPose-L/H at two input sizes and a few recipe changes.
  - We excluded backbones pre-trained on extra pose data.
  - ViTPose with FiDIP-style adaptation was a stretch goal; its status is in Table 6.
  - Only comparisons within MMPose are controlled, because upstream and MMPose also differ in augmentation and training code.
- **Expanded SyRIP.** Its 400 extra real training images are most likely validate500 \ validate100, which is our Val400. Training on them would consume the selection set and change the split, so it is out of scope under the same-split rule. A natural follow-up is to train on the expanded set, select by cross-validation, and report on Test100.
- **Agent-executed work.** An AI agent did most of the execution and made mistakes (Section 9). Tests, cross-checks, pre-registration and provenance records reduce the risk, but no safeguard is complete.

# 8 Lessons learned

**Small-data evaluation.**

- **On 100 test images, evaluation details are as large as method effects.** The id-0 artifact, the evaluator library and the source of the GT boxes each move AP by about FiDIP's whole published gain over fine-tuning (Table 9). Fixing, testing and cross-checking the evaluator *before* running experiments was the most valuable step of the project.
- **Read the code, not only the paper.** Run literally, the README trains plain fine-tuning from random weights and selects on the test set. The paper describes the adversarial update as a gradient reversal layer, but the code implements it as a negative loss weight.
- **Verify the data, not just the code.** File hashes, a near-duplicate search and label/image consistency checks found six issues that would each have silently biased the results:
  - placeholder areas;
  - the id-0 artifact;
  - test images that duplicate training images;
  - selection-set images that duplicate test images;
  - a training file carrying another dataset's labels, shipped with a misplaced test image;
  - unannotated duplicates of validation images in the training folder.

  None of them shows up in a training curve.
- **Read the published numbers diagnostically.** $AP_{50} = AP_{75} = 98.5$ alone shows that all the headroom is at strict thresholds. That pointed the improvement study at localisation precision.

**Working on a shared cluster.**

- **Login nodes are not for work.** Several things failed there:
  - an SSH probe hung behind slow `du` and `module avail` calls;
  - the login node killed `unzip`;
  - reading about 1000 JPEG headers in Python dropped the session twice.

  Using `timeout`, `BatchMode` SSH, and Slurm CPU jobs even for light data work removed these failures.
- **Small tooling mismatches cost time.** macOS ships `openrsync` (protocol 29), which fails against the cluster's rsync 3.2.5 with "unexpected end of file". A ten-line tar-over-ssh script replaced it in both directions.
- **Missing access became provenance.** The lab's project share was not readable, so we downloaded and checksummed the data ourselves.
- **Parallel agents share one filesystem.** A partial extraction was reorganised under a running unzip job, and a file-count check caught the mismatch. Rewriting commit messages changes commit SHAs, so run records also store the tree SHA, which survives the rewrite.
- **Turn policies into guardrails.** "Never select on Test100" became a script that refuses unregistered evaluations and checks checkpoint hashes. "Log as you go" became three live files plus timestamped per-stream notebooks.

### 8.1 Reflection: how the agentic workflow changed the approach

I started by asking the agent for "maximum points + bonus" and for work to begin immediately. Within minutes I interrupted with "Wait" and switched to plan mode. That pause shaped the project. The agent explored the cluster, the upstream code and the dataset read-only and wrote a plan, which I approved at about 19:50, before anything was changed. I supplied the course slides and later the STUDENT_GUIDE, which the agent could not find on its own. These defined deliverables (the three live files, the demos, the over/underfitting analysis), so the plan had to absorb new requirements mid-stream.

**What the agent was good at.** It was good at work that is tedious by hand:

- reading an unfamiliar codebase and finding what decides what a reproduction measures: random initialisation, $\lambda = 0$, and selection on the test set;
- auditing data files line by line, which exposed the placeholder areas, the mislabelled synthetic file and the id-0 artifact;
- running four streams in parallel on the cluster, with tests and notes;
- checking citations against publisher records.

Because checking became cheap, far more of the project went into evaluation conditions than I had expected.

**Where I had to step in.**

- forcing planning before execution;
- providing the course material;
- approving the plan and the parallel-agent setup;
- setting policy, namely a global secrets rule and no AI co-author credit;
- reviewing the results and this report.

The agent's mistakes were caught by checks rather than by luck: estimated timestamps, a wrong claim from its literature scan about automatic position-embedding resizing, and a hung SSH probe. My takeaway is that an agent accelerates execution most when the human fixes the goals, the rules and the standard of evidence first.

# 9 AI-tool use statement

**Tool.** The project used Claude Code [5], Anthropic's agentic coding tool, with the Claude Opus 5.5 model. It ran on the student's laptop with passwordless SSH to Explorer. One orchestrator session delegated work to sub-agents for reproduction, improvement, analysis and the report, each through a written brief. A fresh-context reviewer checks the final state against the rubric. The AI is credited as a tool, not as an author. The student is responsible for the content of the report and the repository.

**What the agent did.**

- explored and planned in plan mode, then executed the approved plan;
- wrote the code: data scripts, Slurm jobs, MMPose configs, the evaluator and its tests, the bootstrap and ensemble tools, the demos, the figure and table generators, and this report's source;
- submitted and monitored cluster jobs, and diagnosed failures;
- audited the upstream code and the data;
- searched the literature and verified the bibliography;
- kept the live files and notebooks current.

**What the human did.**

- set the goal and constraints: the rubric and the course rules;
- stopped the first execution attempt to require a plan, then approved the plan;
- supplied the course slides and the STUDENT_GUIDE;
- asked for the work to be parallelised;
- set policies: a global rule never to print or commit secrets, and no AI co-author attribution in commits or documents;
- reviewed results and documents.

**How outputs were verified.**

(1) **Unit tests:** 24 evaluator tests, plus tests for the bootstrap, ensemble and taxonomy tools (`pytest eval/tests`).
(2) **Cross-check:** our evaluator against the AP that upstream's test script prints for the released checkpoint (Section 3.4).
(3) **Checksums:** SHA256 for every download and checkpoint, recorded in `MANIFEST.md`. Each `run.json` also records the commit and tree SHAs, the Slurm job ID and the checkpoint hash.
(4) **Pre-registration:** enforced by the final-evaluation script.
(5) **No hand-typed results:** every experimental number is read with `\input` from files that `scripts/collect_results.py` generates from the evaluator outputs.
(6) **References:** every reference was checked online against Crossref or the publisher's page; `report/check_bib.py` repeats the DOI check.

(7) **Final review:** by a fresh-context agent that reads the repository and the rubric rather than the workers' summaries.

**Known agent errors, and how they were caught.**

- Estimated timestamps in `AGENT.md`: found by comparing with git commit times, corrected, and a rule added.
- A literature note claiming that MMPretrain resizes ViTPose position embeddings automatically: contradicted by a failing load, and fixed offline.
- A hung SSH probe: fixed with `timeout` and `BatchMode`.
- A race on the shared filesystem: caught by a file-count check.

**Integrity.** No other student's repository was opened or used. Files that the course guide references but upstream lacks were written from the guide's documented interface.

## References


[1] COCO API. https://github.com/cocodataset/cocoapi. Matlab, Python (`pycocotools`) and Lua APIs for the COCO dataset; GitHub repository. Accessed 2026-11-05.

[2] Infant-Pose-Estimation: Official code for "invariant representation learning for infant pose estimation with small data" (FG 2021). https://github.com/ostadabbas/Infant-Pose-Estimation. GitHub repository. Accessed 2026-11-05.

[3] Synthetic and real infant pose (SyRIP) dataset. https://coe.northeastern.edu/Research/AClab/SyRIP/. Dataset download page. Accessed 2026-11-05.

[4] xtcocoapi: Extended COCO-API (`xtcocotools`). https://github.com/jin-s13/xtcocoapi. Version 1.14.3 on PyPI, used by MMPose 1.3.2. Accessed 2026-11-05.

[5] Anthropic. Claude Code. https://code.claude.com/docs/en/overview, 2026. Agentic coding tool; product page https://claude.com/product/claude-code. Accessed 2026-11-05.

[6] S. Bose, H. Dela Cruz, A. Dutta, E. Kokkoni, K. Karydis, and A. K. Roy-Chowdhury. Leveraging synthetic adult datasets for unsupervised infant pose estimation. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 5562–5571, Nashville, TN, USA, June 2025. IEEE. doi:10.1109/cvprw67362.2025.00553.

[7] X. Cao, X. Li, L. Ma, Y. Huang, X. Feng, Z. Chen, H. Zeng, and J. Cao. AggPose: Deep aggregation vision transformer for infant pose estimation. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22)*, pages 5045–5051, Vienna, Austria, July 2022. International Joint Conferences on Artificial Intelligence Organization. doi:10.24963/ijcai.2022/700. Special Track on AI for Good.

[8] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993. ISBN 978-0-412-04231-7. doi:10.1201/9780429246593.

[9] F. Gama, M. Mísař, L. Navara, S. T. Popescu, and M. Hoffmann. Automatic infant 2D pose estimation from videos: Comparing seven deep neural network methods. *Behavior Research Methods*, 57(10):280, 2025. doi:10.3758/s13428-025-02816-x.

[10] Y. Ganin and V. Lempitsky. Unsupervised domain adaptation by backpropagation. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1180–1189, Lille, France, July 2015. PMLR. URL https://proceedings.mlr.press/v37/ganin15.html.

[11] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59): 1–35, 2016. URL https://jmlr.org/papers/v17/15-239.html.

[12] A. Grafton, J. M. Warnecke, M. Li, E. He, L. Thomson, K. Beardsall, and J. Lasenby. Neonatal pose estimation in the unaltered clinical environment with fusion of RGB, depth and IR images. *npj Digital Medicine*, 8(1):539, 2025. doi:10.1038/s41746-025-01929-z.

[13] N. Hesse, S. Pujades, J. Romero, M. J. Black, C. Bodensteiner, M. Arens, U. G. Hofmann, U. Tacke, M. Hadders-Algra, R. Weinberger, W. Müller-Felber, and A. S. Schroeder. Learning an infant body model from RGB-D data for accurate full body motion analysis. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2018: 21st International Conference, Granada, Spain, September 16–20, 2018, Proceedings, Part I*, volume 11070 of *Lecture Notes in Computer Science*, pages 792–800, Cham, 2018. Springer. doi:10.1007/978-3-030-00928-1_89.

[14] N. Hesse, C. Bodensteiner, M. Arens, U. G. Hofmann, R. Weinberger, and A. S. Schroeder. Computer vision for medical infant motion analysis: State of the art and RGB-D data set. In *Computer Vision – ECCV 2018 Workshops: Munich, Germany, September 8–14, 2018, Proceedings, Part VI*, volume 11134 of *Lecture Notes in Computer Science*, pages 32–49, Cham, 2019. Springer. doi:10.1007/978-3-030-11024-6_3.

[15] J. Huang, Z. Zhu, F. Guo, and G. Huang. The devil is in the details: Delving into unbiased data processing for human pose estimation. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5699–5708, Seattle, WA, USA, June 2020. IEEE. doi:10.1109/cvpr42600.2020.00574.

[16] X. Huang, N. Fu, S. Liu, and S. Ostadabbas. Invariant representation learning for infant pose estimation with small data. In *2021 16th IEEE International Conference on Automatic Face and Gesture Recognition (FG 2021)*, pages 1–8, Jodhpur, India, Dec. 2021. IEEE. doi:10.1109/fg52635.2021.9666956.

[17] X. Huang, M. Wan, L. Luan, B. Tunik, and S. Ostadabbas. Computer vision to the rescue: Infant postural symmetry estimation from incongruent annotations. In *2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1909–1917, Waikoloa, HI, USA, Jan. 2023. IEEE. doi:10.1109/wacv56688.2023.00195.

[18] Z. Hui, J. Wang, X. Dong, G. Zhang, X. Wang, J. Peng, Q. Liu, X. Yuan, Y. Zhang, J. O. Yin, W. Ma, Q. Lyu, S. N. Lupolt, K. E. Nachman, A. Kortylewski, and A. Yuille. InfantNet: A large scale dataset for infant body pose and shape estimation. arXiv preprint arXiv:2610.40467, 2026. URL https://arxiv.org/abs/2610.40467.

[19] L. Jahn, S. Flügge, D. Zhang, L. Poustka, S. Bölte, F. Wörgötter, P. B. Marschik, and T. Kulvicius. Comparison of marker-less 2D image-based methods for infant pose estimation. *Scientific Reports*, 15(1): 12148, 2025. doi:10.1038/s41598-025-96206-0.

[20] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *Computer Vision – ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part V*, volume 8693 of *Lecture Notes in Computer Science*, pages 740–755, Cham, 2014. Springer. doi:10.1007/978-3-319-10602-1_48.

[21] MMPose Contributors. OpenMMLab pose estimation toolbox and benchmark. https://github.com/open-mmlab/mmpose, 2020.

[22] Northeastern University Research Computing. NURC RTD: HPC documentation for the Northeastern University Explorer Cluster. https://rc-docs.northeastern.edu/en/latest/, 2026. Accessed 2026-11-05.

[23] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, Salt Lake City, UT, June 2018. IEEE. doi:10.1109/cvpr.2018.00474.

[24] K. Sun, B. Xiao, D. Liu, and J. Wang. Deep high-resolution representation learning for human pose estimation. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5686–5696, Long Beach, CA, USA, June 2019. IEEE. doi:10.1109/cvpr.2019.00584.

[25] G. Varol, J. Romero, X. Martin, N. Mahmood, M. J. Black, I. Laptev, and C. Schmid. Learning from synthetic humans. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4627–4635, Honolulu, HI, July 2017. IEEE. doi:10.1109/cvpr.2017.492.

[26] B. Xiao, H. Wu, and Y. Wei. Simple baselines for human pose estimation and tracking. In *Computer Vision – ECCV 2018: 15th European Conference, Munich, Germany, September 8–14, 2018, Proceedings, Part VI*, volume 11210 of *Lecture Notes in Computer Science*, pages 472–487, Cham, 2018. Springer. doi:10.1007/978-3-030-01231-1_29.

[27] Y. Xu, J. Zhang, Q. Zhang, and D. Tao. ViTPose: Simple vision transformer baselines for human pose estimation. In *Advances in Neural Information Processing Systems*, volume 35, pages 38571–38584. Curran Associates, Inc., 2022. doi:10.52202/068431-2795. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/fbb10d319d44f8c3b4720873e4177c65-Abstract-Conference.html.

[28] F. Zhang, X. Zhu, H. Dai, M. Ye, and C. Zhu. Distribution-aware coordinate representation for human pose estimation. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7091–7100, Seattle, WA, USA, June 2020. IEEE. doi:10.1109/cvpr42600.2020.00712.

[29] Z. Zhou, Z. Jiang, W. Chai, C.-Y. Yang, L. Li, and J.-N. Hwang. Efficient domain adaptation via generative prior for 3D infant pose estimation. In *2024 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, pages 51–59, Waikoloa, HI, USA, Jan. 2024. IEEE. doi:10.1109/wacvw60836.2024.00013.

# A Reproduction commands

All commands run from the repository root. Cluster commands run in `/scratch/purohitham.v/fidip/repo`, which is kept in sync with `scripts/sync_to_cluster.sh`. `README.md` has the full, current sequence, and each run's exact command is recorded in its `run.json` and in `MANIFEST.md`.

```
# 0. access (SKILL.md): passwordless key, host alias "explorer"; never compute on the login node
ssh -o BatchMode=yes -o ConnectTimeout=15 explorer 'hostname'
bash scripts/sync_to_cluster.sh # tar-over-ssh push of code (never deletes)
# 1. environments (CPU jobs)
sbatch scripts/slurm/build_env_fidip.sbatch # pip-only py3.12 / torch 2.3.1 cu121
sbatch scripts/slurm/build_env_mmpose.sbatch # py3.10 / torch 2.1.2 / mmpose 1.3.2
sbatch scripts/slurm/build_env_analysis.sbatch # numpy 1.26.4 + pycocotools 2.0.8
# 2. data: download, SHA256 verify, unzip on a compute node, upstream layout, split files
sbatch scripts/slurm/get_data.sbatch
# 3. checkpoints (SHA256 in MANIFEST.md section 4)
sbatch scripts/slurm/get_ckpts_mmpose.sbatch; sbatch scripts/slurm/prep_ckpts_mmpose.sbatch
# 4. FiDIP retrain = the README command plus the two overrides it needs (one seed shown)
python tools/train_adaptive_model_hrnet.py \
  --cfg experiments/coco/hrnet/w48_384x288_adam_lr1e-3_infant.yaml \
  TRAIN.CHECKPOINT models/coco/w48_384x288.pth TRAIN.LAMBDA 0.0005 # 0.0 = fine-tune ablation
# 5. MMPose run (Val400 selection inside; never touches Test100)
sbatch --export=ALL,RUN_ID=<RUN_ID>,CONFIG=mmpose_exps/configs/<config>.py,SEED=1 \
  scripts/slurm/train_mmpose.sbatch
# 6. final Test100 evaluation of a pre-registered run (refuses otherwise)
sbatch --export=ALL,RUN_ID=<RUN_ID> scripts/slurm/final_test_eval_mmpose.sbatch
# 7. metrics, CIs, tables, figures, report
python eval/eval_coco.py --gt <split>.json --pred runs/<RUN_ID>/preds_test100.json \
  --out runs/<RUN_ID>/metrics_test100.json --per-image runs/<RUN_ID>/per_image_test100.json
python eval/bootstrap_ci.py multiseed --gt <test100>.json --a <A_s1> <A_s2> <A_s3> --b <B> --out ci.
    json
bash scripts/sync_results.sh && python scripts/collect_results.py # -> report/generated/*.tex
bash report/build.sh # -> report/FiDIP_Report.pdf
```

# B Upstream traps and patches

Six details of the upstream code matter for anyone reproducing it:

(i) The literal README command is plain fine-tuning from random weights (Section 2.3). We pass `TRAIN.CHECKPOINT` and `TRAIN.LAMBDA` on the command line rather than editing code.

(ii) The loader always builds the adult `train_pre_infant` set, even when `PRE_EPOCH` is 0.

(iii) `lib/dataset/syrip.py` skips evaluation for any split whose name contains "test".

(iv) `tools/test_adaptive_model.py` loads weights with `strict=False`, so a key-prefix mismatch would silently leave the network at its initialisation. We added a strict-load check.

(v) The compiled NMS extensions are never used, so `make` can be skipped.

(vi) For ViTPose at 384×288, MMPretrain infers a *square* grid from the 192 position embeddings of the 16×12 COCO checkpoint and fails. We resized them offline (bicubic, 16×12 → 24×18, `mmpose_exps/tools/resize_vit_pos_embed.py`).

**Table 11:** Patches to the vendored upstream code (`third_party/Infant-Pose-Estimation`), one commit each, generated from `MANIFEST.md` §6.

| ID | File | Change | Why |
|---|---|---|---|
| TBD | TBD | TBD | TBD |

# C Evaluator tests

`python -m pytest eval/tests` runs the evaluator's unit tests on mock data; no SyRIP files are needed. They check:

- that the GT used as the prediction gives AP = AR = 1;
- the id-0 artifact (`paper` versus `fixed`) and the exact `paper`-protocol ceiling formula;
- monotone degradation of AP, OKS and PCK under increasing keypoint noise;
- bit-identical metrics across twelve prediction encodings;
- that unknown images are dropped and missing predictions count as misses;
- the rescoring invariances: identical results for files already scored by the rule, and indifference to random file scores, monotone score transforms and confidence scaling;
- that the rescoring rule equals upstream's loop;
- the constant-area guard;
- that per-image OKS equals `COCOeval`'s internal values;
- exact per-keypoint errors and PCK on constructed shifts;
- that per-threshold AP averages back to AP, $AP_{50}$ and $AP_{75}$;
- that OKS-NMS keep-lists equal upstream `nms.py` on random instance sets;
- the command line end to end.

The bootstrap, ensemble, taxonomy and analysis tools have their own tests in the same folder.

# D Hyperparameters

# E Per-keypoint results

# F Compute used

The project used TBD Slurm jobs and TBD GPU-hours in total, including TBD training runs (Table 14; accounting from `sacct`). GPU jobs ran on H200 and A100 nodes of the `gpu` and `sharing` partitions. Data preparation, environment builds and analysis ran as CPU jobs.

# G Provenance: from a number to its run

To trace any number in this report:

1. `MANIFEST.md` §8 maps each reported number to a run ID and a metrics file.
2. The run's row in `results/registry.csv` gives the Slurm job ID, config, seed, GPU, git SHA and checkpoint SHA256.
3. The run directory `runs/<RUN_ID>/` on `/scratch` (with persistent copies in `~/fidip_artifacts/`) holds `run.json`, the logs, the predictions and the `eval_coco.py` output. Its `gt.sha256` and

**Table 12:** Resolved training and inference settings. Upstream values come from the released yaml and code at `a76b4f4`, plus the two command-line overrides. MMPose values come from the dumped configs of the selected runs.

| Setting | FiDIP retrain (upstream code) | MMPose control (HRNet-W48) | ViTPose-L / H |
|---|---|---|---|
| Initial weights | COCO DarkPose `w48_384x288.pth` | MMPose HRNet-W48 DARK COCO (`39c3c381`) | MMPose ViTPose COCO 256×192 (`53609f55` / `e32adcd4`); pos. embeddings resized for 384×288 |
| Input / heatmap | 384×288 / 96×72, $\sigma = 3$ | 384×288 / 96×72 (DARK) | 256×192 or 384×288 (UDP) |
| Optimiser | Adam, LR $10^{-4}$ constant, no weight decay | Adam, LR **TBD** | AdamW, LR **TBD**, wd 0.1, layer decay 0.8 / 0.85, clip 1.0, drop-path 0.5 / 0.55 |
| Batch / epochs | 20 / 20 | 32 / **TBD** | 64 / **TBD** |
| Schedule | constant (steps at 40, 200 never reached) | warm-up 1 epoch, ×0.1 late (**TBD**) | same as control |
| Domain adaptation | $\lambda = 5{\cdot}10^{-4}$ (FiDIP) or 0; $D$: Adam $10^{-4}$ | none | none |
| Frozen layers | first 109 parameter tensors | **TBD** | **TBD** |
| Augmentation | flip 0.5; rotation $\mathcal{N}(0, 45°)$ clipped to ±90° w.p. 0.6; scale ±35%; half-body 0.3 | MMPose COCO default (**TBD**) | same as control |
| Real : synthetic | balanced sampler, 1:1 (real oversampled ×5) | 1:1 (`CombinedDataset`, real ×5) | same |
| Test time | GT boxes, flip, DARK (blur kernel 11) | GT boxes, flip, DARK | GT boxes, flip, UDP |
| Selection | best Val400 AP epoch | best Val400 AP epoch | best Val400 AP epoch |

`pred.sha256` fields pin the exact files.

4. `report/generated/*.tex` is regenerated from those metrics by `scripts/collect_results.py`; the report only reads it with `\input`.

Decisions (D1, D2, ...) are dated and explained in `AGENT.md`, and command-level detail is in `notebook/<stream>.md`.

# H Rubric traceability

**Table 13:** Per-keypoint accuracy on Test100: median error as a percentage of $\sqrt{wh}$ of the GT box, and PCK with the same normaliser, over labelled joints only.

| | Released FiDIP (R001) | | | Best configuration | | |
|---|---|---|---|---|---|---|
| Keypoint | med. err. (%) | PCK@.05 | PCK@.1 | med. err. (%) | PCK@.05 | PCK@.1 |
| Head (nose, eyes, ears) | TBD | TBD | TBD | TBD | TBD | TBD |
| Shoulders | TBD | TBD | TBD | TBD | TBD | TBD |
| Elbows | TBD | TBD | TBD | TBD | TBD | TBD |
| Wrists | TBD | TBD | TBD | TBD | TBD | TBD |
| Hips | TBD | TBD | TBD | TBD | TBD | TBD |
| Knees | TBD | TBD | TBD | TBD | TBD | TBD |
| Ankles | TBD | TBD | TBD | TBD | TBD | TBD |
| All | TBD | TBD | TBD | TBD | TBD | TBD |

**Table 14:** Compute by work stream, from `sacct`.

| Stream | Slurm jobs | GPU types | GPU-hours | CPU-only jobs |
|---|---|---|---|---|
| Reproduction | TBD | TBD | TBD | TBD |
| Improvement | TBD | TBD | TBD | TBD |
| Analysis | TBD | TBD | TBD | TBD |
| Total | TBD | | TBD | TBD |

**Table 15:** Where each grading item is addressed.

| Rubric item | Report | Repository |
|---|---|---|
| Environment setup, data preparation | Sections 3.2 and 3.3 | `envs/`, `scripts/get_data.sh`, `scripts/make_splits.py` |
| Working inference and evaluation pipeline | Sections 3.4, 4.1 and 6.4 | `eval/eval_coco.py`, `eval/tests/`, `tools/demo.py` |
| Reproducible instructions, code, configs | Appendices A, D and G | `README.md`, `MANIFEST.md`, `scripts/slurm/`, `mmpose_exps/configs/` |
| Correct quantitative evaluation | Sections 3.4 and 6.1 | `eval/`, `results/PREREGISTRATION.md` |
| Comparison with the published baseline | Tables 2, 3 and 7; box in Section 5.4 | `results/registry.csv` |
| Visualisations, success/failure analysis | Section 6 | `eval/visualize.py`, `eval/analyze.py`, `eval/taxonomy.py` |
| Organisation; methods and choices | Sections 1 to 3 and 5.1 | `README.md` |
| Limitations, lessons, references, AI use | Sections 7 to 9; references | `AGENT.md`, `report/refs.bib` |
| Bonus: improvement, ablation, repeats, conditions | Section 5; Tables 4 to 7 | `results/PREREGISTRATION.md`, `MANIFEST.md` |
| Live `SKILL.md` / `AGENT.md` / `MANIFEST.md` | Sections 3.1 and 9 | repository root |